



Ingeniería de Software I

Estudiante: Carlos Daniel Flores

Matrícula: 100081538

Sistema digital trabajado:

ClinicaSystem aplicación de escritorio (C# WinForms) para la gestión de una clínica: administra pacientes, médicos, citas, historial médico y usuarios, con control de acceso basado en roles (Administrador, Recepcionista, Médico) y base de datos SQL Server

Estilo arquitectónico seleccionado: Arquitectura por capas (Layered Architecture)

Se seleccionó la arquitectura por capas como estilo arquitectónico para ClinicaSystem, organizada en cuatro capas principales: Presentación, Lógica de negocio, Acceso a datos y Base de datos, con un servicio transversal de Autenticación.

Justificación:

Este estilo se eligió porque refleja fielmente la estructura que el sistema ya tiene implementada:

- La capa de **Presentación** corresponde a los formularios Windows Forms (FrmLogin, FrmMenuPrincipal, FrmPacientes, etc.).
- La capa de **Lógica de negocio** aplica las reglas del dominio clínico (validación de cédula única, disponibilidad de citas, etc.).
- La capa de **Acceso a datos** está representada por los Repositories (PacienteRepository, MedicoRepository, CitaRepository, HistorialMedicoRepository, UsuarioRepository) junto con el helper centralizado DbHelper.
- La capa de **Base de datos** es SQL Server (ClinicaDB).
- La **Autenticación** se maneja como un servicio transversal porque interactúa tanto con la Presentación (login, control de botones por rol) como con la Lógica de negocio (autorización de operaciones), sin pertenecer exclusivamente a una sola capa.

Tabla de componentes para la arquitectura de ClinicaSystem.

Componente	Responsabilidad	Relación con otros componentes
Presentación (Formularios WinForms: FrmLogin, FrmMenuPrincipal, FrmPacientes, FrmMedicos, FrmCitas, FrmHistorialMedico, FrmUsuarios)	Interfaz de usuario: captura datos, muestra información, valida entradas a nivel de formulario, restringe botones/menús según el rol del usuario	Envía solicitudes a la capa de Lógica de negocio; consulta a Autenticación para validar sesión y permisos
Autenticación (servicio transversal — login, hashing PBKDF2+salt, bloqueo por intentos fallidos)	Verificar credenciales, gestionar sesión activa, aplicar control de acceso basado en roles (Administrador, Recepcionista, Médico)	Se conecta con Presentación (login, mostrar/ocultar opciones por rol) y con Lógica de negocio (autorizar operaciones antes de ejecutarlas)
Lógica de negocio (reglas de validación: cédula única, disponibilidad de citas, validación de historial médico)	Aplicar reglas del dominio clínico, coordinar operaciones entre Presentación y Acceso a datos, validar antes de persistir	Recibe solicitudes de Presentación; usa Acceso a datos para leer/escribir; consulta Autenticación para permisos
Acceso a datos (Repositories: PacienteRepository, MedicoRepository, CitaRepository, HistorialMedicoRepository, UsuarioRepository + DbHelper + DatosException)	Ejecutar operaciones CRUD contra la base de datos, traducir errores SQL a mensajes en español, centralizar el manejo de conexiones	Recibe solicitudes de Lógica de negocio; se comunica directamente con la Base de datos
Base de datos (SQL Server — ClinicaDB)	Almacenamiento persistente de pacientes, médicos, citas, historial médico y usuarios	Solo accesible a través de la capa de Acceso a datos

Decisión arquitectónica

Se decidió estructurar ClinicaSystem bajo una arquitectura por capas con Autenticación como servicio transversal, dado que esta organización ya está presente de forma natural en el código (formularios, repositorios, DbHelper y base de datos SQL Server), y porque separa claramente las responsabilidades del sistema: la interfaz no accede directamente a la base de datos, y la lógica de negocio no depende de detalles de la interfaz.

El principio de diseño aplicado es la **separación de responsabilidades (Separation of Concerns)**, reforzado por el principio de **responsabilidad única**: cada capa tiene una única razón para cambiar (la Presentación cambia si cambia la interfaz, el Acceso a datos cambia si cambia el motor de base de datos, etc.), lo que facilita el mantenimiento y las pruebas de cada capa por separado.

Como limitación, este estilo introduce cierto acoplamiento vertical entre capas consecutivas (por ejemplo, un cambio en la estructura de la base de datos puede requerir ajustes en Acceso a datos y, en algunos casos, en Lógica de negocio). Además, al no existir todavía una capa de servicios independiente (API), el sistema depende de que la Presentación y la Lógica de negocio se ejecuten en el mismo entorno de escritorio, lo que limita una futura migración hacia una arquitectura distribuida o web sin una refactorización considerable.

Tarjeta en GitHub Projects

Tarjeta "**Definición de arquitectura inicial**" (#9) registrada en el proyecto *ClinicaSystem* del repositorio Carlosrose07/ClinicaSystem, con la justificación técnica y el enlace al diagrama arquitectónico.



Diagrama arquitectónico (Visual Paradigm Online): <https://online.visual-paradigm.com/share.jsp?id=343639393131332d35>

Diagrama arquitectónico

